Topic 08: PROBLEM SOLVING

Read the following case studies

Case study 01: Mobile payment system in Tanzania

Problem (Past Situation):

- A few decades ago, many people in Tanzania did not have access to banking services.
- Financial transactions (like sending, receiving, or saving money) were difficult.
- This caused insecurity and inefficiency in financial services for both individuals and businesses.

Solution (Present Situation):

- **Telecommunication companies** introduced **mobile money services** (e.g., M-Pesa, Mixx by Yas, Airtel Money, and Halopesa).
- These allow people to transfer, deposit, and withdraw money easily using mobile phones.

Impact (Result):

- Improved money flow and access to financial services.
- Increased safety and efficiency in transactions.
- Enhanced **standard of living** and **economic participation** for many Tanzanians.

Case Study 02: Viewing Examination Results I n Tanzania

Background (Problem):
In the past, students in Tanzania faced challenges accessing their examination results.

- Results were posted on school noticeboards or published in newspapers.
- This process was slow, sometimes unreliable, and not accessible to students in remote areas.
- Many students had to travel long distances or wait several days to know their results.

Solution (Use of ICT):
The **National Examination Council of Tanzania (NECTA)** introduced **online result systems**.

- Students can now view their results using the **NECTA website**
- Results for **PSLE**, **Form Two**, **Form Four**, and **Form Six** can be accessed using a computer or smartphone with internet connection.
- Some mobile networks and schools also provide **SMS result-checking services**.

Impact (Result):

- Students and parents can access results **quickly and easily** from anywhere.
- Reduced overcrowding and delays at schools.
- Increased **transparency** and **efficiency** in result dissemination.
- Promotes digital literacy and trust in ICT systems.

Case Study 03: Receiving and Paying Utility Bills in Tanzania

Background (Problem):
In the past, Tanzanians paid their **electricity**, **water**, and **other utility bills** manually.

- Customers had to visit company offices (like TANESCO or DAWASA) and queue for hours to make payments.
- Delays, errors, and loss of receipts were common.
- Some customers faced **service disconnections** due to late or missed payments caused by these challenges.

Solution (Use of ICT):
With the development of **Information and Communication Technology (ICT)**, most utility companies in Tanzania adopted **electronic billing and payment systems**, including:

- **Mobile money services** such as **M-Pesa, Mixx by Yas, Airtel Money, and HaloPesa**.
- **Online platforms and apps** (e.g., TANESCO e-payment, DAWASA online billing system).
- **Automated messages (SMS or email)** sent to customers showing their bill amount and payment deadlines.

Customers can now:

1. Receive billing notifications instantly.
2. Pay bills from their phones or computers anytime, anywhere.
3. Receive electronic receipts automatically after payment.

Impact (Result):

- Saves time and reduces the need to travel or queue.
- Promotes **cashless transactions** and financial security.
- Reduces errors in billing and record-keeping.
- Improves efficiency and customer satisfaction for both users and service providers.
- Encourages the use of **ICT for e-governance and digital services**.

MEANING OF PROBLEM SOLVING

Problem solving is the process of studying, analyzing, and creating a systematic method for finding a solution to a problem, and then applying that method to reach the desired result.
In Computer Science, it involves using logical thinking and computer programs to solve real-life challenges.
This approach of identifying a problem, understanding its cause, designing a way to solve it, and applying that solution is what defines problem solving.
Problem-solving skills are useful in all fields, but in this topic, the main focus is on solving problems using computers.

IMPORTANCE OF STUDYING PROBLEM SOLVING

Problem solving is one of the most important skills in Computer Science.
It forms the foundation for almost every aspect of computer science and software development.

The importance of problem solving skills in developing a program is as follows:

(i) It helps in designing and optimizing algorithms, making them run efficiently and use resources effectively.
(ii) It helps learners develop the ability to break complex tasks into smaller, manageable parts and solve them step by step.

TYPES OF PROBLEMS

There are so many problems in our daily lives, but all these problems can be grouped into two categories:

(i) **Problems that require algorithmic solutions.**
These are problems that can be solved by following a series of clear steps or actions.
Their solutions can be found by identifying the correct sequence of operations to perform.

Example of such problems are finding an average of marks scored, sorting numbers, calculating utility bills and finding the largest value out of many numbers

Computers are mainly built to deal with algorithmic solutions, which are often difficult or more time-consuming for humans

Problem solving therefore, involves preparing instructions that must be followed to produce the best results

The act of preparing this solution is known as programming

The outcomes of the completed computer assisted answer of programming is a program

A program is a set of instructions that make up the solution after they have been coded into a particular computer language.

(ii) **Problems that require heuristic solutions.**
These are problems that do not have a definite procedure for reaching a solution.
Instead, they require experience, judgment, or creativity to find the best possible answer.

Their solutions are not always exact but are reached through judgment or estimation.
For example, diagnosing a medical problem, designing a building, or playing chess may require heuristic methods.

The field of computers that deals with heuristic types of problems is called artificial intelligence

Artificial Intelligence (AI) systems use stored knowledge and past experience to make intelligent decisions, similar to how humans solve problems

CHALLENGES OF PROBLEM SOLVING

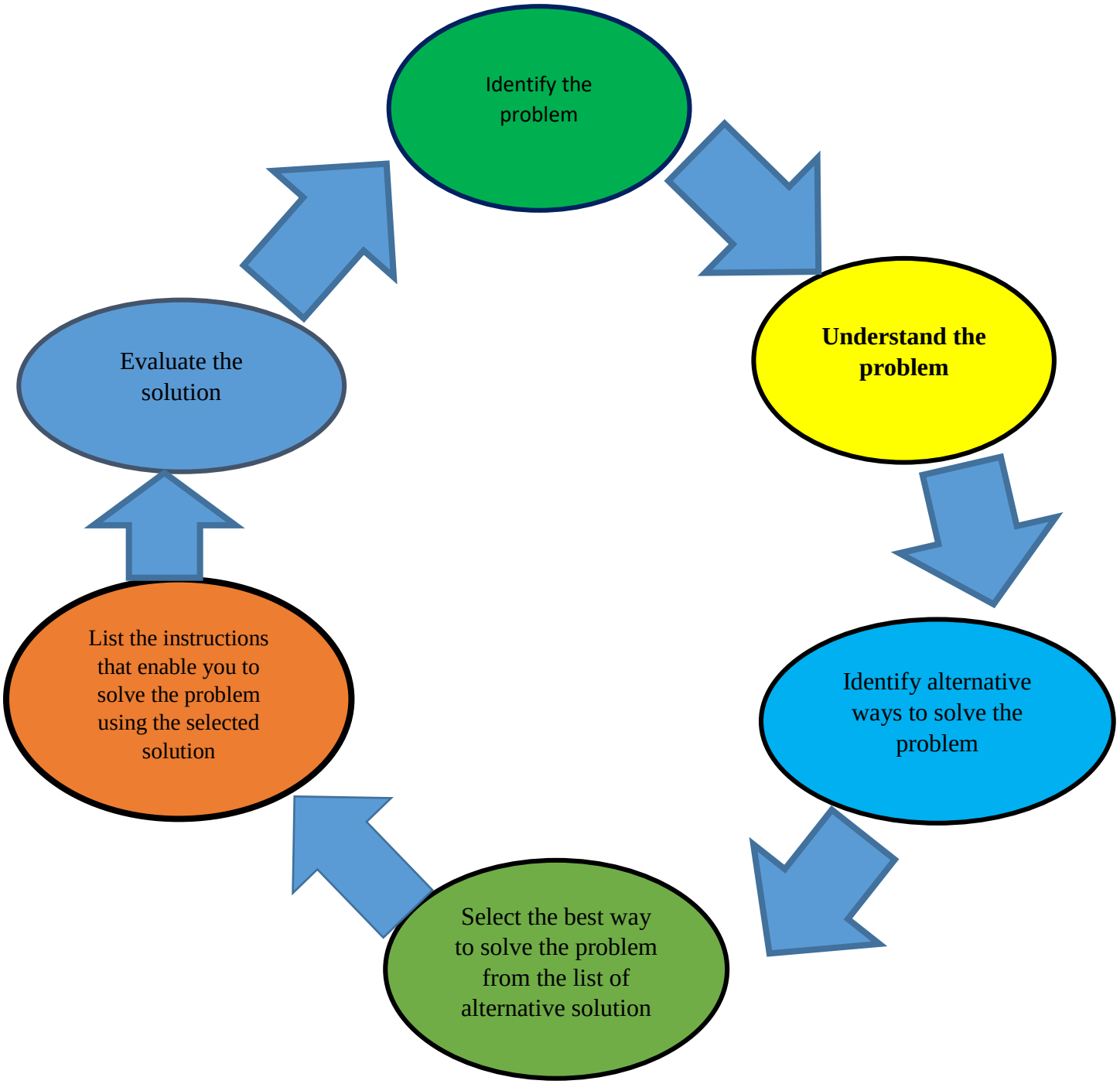
Although many problems can be solved, people often face difficulties during the process.
Common challenges include:

i) **Lack of knowledge and skills** – Not knowing how to approach or solve a problem effectively.
ii) **Fear of making mistakes** – This leads to delays and failure to make decisions.
iii) **Failure to complete all steps** – Some problems remain unsolved because one or more steps in the process are skipped.
iv) **Poor problem definition** – If a problem is not clearly defined, it may be misunderstood and not properly solved.
v) **Poor logical sequencing** – Failing to arrange steps in the correct order can lead to wrong results.
vi) **Carelessness or haste** – Working too quickly without proper checking may result in errors.
vii) **Failure to evaluate solutions** – Not testing or reviewing the solution to confirm its correctness.

STEPS OF PROBLEM SOLVING

Good problem solving skills can save a lot of time and resources. Effective problem solvers are adapt at dissecting an issue to understand its root cause, thus reducing the time spent on trial and error. This efficiency means projects move faster, and the results are archived in due time

There are six steps to solve both algorithmic and heuristic problems as shown below



It is important to note that problem solving is a continuous process. Once the solution is implemented, another problem usually arises, and the cycle starts again

Step 1: Identify the Problem.
This step involves recognizing that a problem exists and clearly defining what it is. You must understand what you are going to solve before starting. If you don't know the problem, you can't solve it. Ask yourself, "What am I going to solve?"

Step 2: Understand the Problem
In this step, you study the problem in detail, including its causes, effects, and possible solutions. You need to ask yourself several questions to understand what is involved. By analysing the problem, you can know what needs to be done and what resources or knowledge are required to solve it.

Step 3: Identify Alternative Ways to Solve the Problem
A problem can have more than one possible solution. This step involves finding and analysing different ways or strategies that can be used to solve it. You should list all possible alternatives before choosing the best one.

Step 4: Select the Best Way to Solve the Problem.
After examining all the possible solutions, choose the most effective, efficient, and feasible one. The selected solution should best address the problem using the available resources.

You need to identify and evaluate the advantages and disadvantages of each possible solution before selecting the best one. To do this you need to select criteria for the evaluation

These criteria will serve as the guidelines for evaluating each solution

Step 5: Implement the solution.
This step involves *listing instructions that enable you to solve the problem using the selected solution.* These instructions are then followed one after another to carry out the solution. Implementation may include writing a program or following a set of procedures to achieve the desired result.

Step 6: Evaluate the Solution.
After implementation, you assess whether the problem has been solved successfully. You check if the results meet the expected goals. If the problem still exists, you may go back to earlier steps and make improvements or choose another solution.

ALGORITHMS

Consider the scenario below
Scenario 01: Performing everyday activities
Every day, Kobe performs activities by following steps. Examples of activities he performs include getting ready for body exercise, preparing breakfast, riding a bicycle, wearing shoes and washing utensils. To complete each activity he follows a sequence of steps. Examples of steps he uses to ride a bicycle are to:

a) Remove the bicycle from the stand
b) Sit on the seat of the bicycle
c) Start peddling
d) Use breaks whenever needed
e) Stop on reaching the destination

Questions

1. Explain what will happen if Kobe skips step (c) in the scenario.
   Answer: The bicycle will not move and therefore will not reach his destination
2. What is the importance of following all the steps without skipping any?
   Answers
   (i) **Effectiveness:** Each step contributes to achieving the goal. Missing one step may make the goal impossible to reach.
   (ii) **Safety:** Some steps are there to prevent harm or accidents. Skipping them can put Kobe at risk.
   (iii) **Efficiency and Order:** Steps are designed to be performed in a logical sequence. Following them prevents unnecessary effort or confusion.
   (iv) **Reliability:** Consistently following all steps ensures the same positive result every time.
   (v) **Skill Development:** Doing activities step by step improves discipline, focus, and the ability to perform tasks correctly in the future

MEANING OF ALGORITHMS

An algorithm is a set of steps leading to the solution of a given problem.

OR

An **algorithm** is a **step-by-step procedure or set of instructions** designed to solve a specific problem or perform a particular task.

It is method of representing the systematic and logical procedure for solving a problem.

The steps in scenario 01 are example of an algorithm for the smooth riding of bicycle. Thus, from the scenario, a set of procedure is required to be performed for a given task.

The steps to solve a problem like those performed by Kobe form an algorithm. Before developing an algorithm for solving a given problem think of what the output will be when such an algorithm is implemented. Other examples of algorithms include steps of cooking tea and finding the greatest common factor (GCF) of two numbers such as 48 and 18

IMPORTANCE OF AN ALGORTHIM

(i) It is essential to enhancing a computer program according to the available resources
(ii) It enables you to solve a problem through a better approach
(iii) It saves for the best program speed and desired minor memory consumption
(iv) It emphasizes solving problems in a straightforward manner

CHARACTERISTICS OF SUITABLE ALGORITHM

(i) ***Well-defined input***: An algorithm requires some input values, such input values can be zero or more but only a finite number of inputs are required
(ii) ***Well defined output***: The algorithm must give an output (one or more) once executed. The output produced should have a specific relationship with the input
(iii) ***Unambiguity***: The algorithm's instructions should be clear, precise, and well communicated
(iv) ***Effectiveness:*** Each instruction in an algorithm should be adequate. That is an algorithm should be designed in such a way that it correctly achieves its purpose and does exactly it was intended to be done in a time length of time
(v) ***Programming language independence:*** The algorithm should not depend on a particular programming language for its implementation. That is once presented, it should always provide the same result regardless of the programming language used to write the program
(vi) ***Finiteness/ Terminator:*** The algorithm must terminate after a finite number of steps, i.e. it cannot run forever. That is for all different cases, the written algorithm should provide the result within a finite number of steps

QUALITIES OF A GOOD ALGORITHM

(i) ***Memory utilization:*** The best algorithm is the one that will ensure that a program consumes the least memory

(ii) ***Efficiency***: The best algorithm enables a computer program to produce results while using less processing power in a short time

(iii) ***Cost-effective:*** The best algorithm is the one that is simple, straightforward and produced using minimal cost.

(iv) ***Accuracy:*** The best algorithm may provide more accurate results than other existing algorithms.

RECOMMENDATIONS FOR WRITING A SUITABLE COMPUTER ALGORITHMS

The following are recommendations for writing a good algorithm:

(i) The instructions in the algorithms should be clear and precise to the programmer
(ii) Write steps of algorithms in logical sequence
(iii) Avoid infinite repetitions of instructions
(iv) Phrases or words used sound like typical natural language
(v) Make sure that the algorithms will finally terminate.

USES OF ALGORITHMS

An algorithm aims at increasing the reliability, accuracy, and efficiency of obtaining solutions. Algorithms are helpfully for small and complicated problems

Algorithms are used to

(i) Improve problem-solving by guiding how tasks are done.
(ii) Increase efficiency in performing operations.
(iii) Ensure accuracy and correctness of results.
(iv) Provide a logical sequence of steps for solving a problem.
(v) Simplify complex problems into smaller, manageable steps.

PRINCIPLES OF ALGORITHM DESIGN

Some principles that guide the design of algorithm include:

(i) ***Correctness:*** Any valid input to the algorithm should produce the correct output and solve intended problem without producing errors.

(ii) ***Efficiency:*** Algorithms should be efficient in terms of time and space. This means that the algorithms must work within a reasonable amount of time. Also , it should not employ large amounts of memory or resources

(iii) ***Input and output:*** Algorithms should have well defined input and output specifying the expected input and output type to be produced
(iv) ***Determinism:*** The algorithm should produce the same output for the same input and not something that unexpected
(v) ***Finiteness:*** Steps that are followed in algorithms must end. They should terminate and produce the desired output after several computations
(vi) ***Scalability:*** Algorithms should be capable of handling larger problem sizes such that when input size increases the performance should not decrease
(vii) ***Modularity:*** Algorithms should be put into smaller sub problems to ensure the readability of code when implemented.

CONSTRUCTION OF ALGORITHMS

Algorithms are used **knowingly or unknowingly** in daily activities.

They involve performing tasks using **defined steps**.

Examples include **making decisions, selecting and arranging items, and classifying things** using a given rule.

Common real-life examples are **cooking**, **assembling a computer**, **starting a computer**, **sorting examination papers**, and **finding the shortest route**.

An algorithm provides a **systematic and logical way** to complete tasks efficiently

For example, you can write an algorithm to make a cup of tea as follows

(i) Get a teabag, cup, kettle, water, sugar and teaspoon
(ii) Put some water in the kettle
(iii) Boil the water in the kettle
(iv) Put the teabag in the cup
(v) Pour some boiled water into the cup
(vi) Add a teaspoon of sugar to the cup
(vii) Stir the tea using a teaspoon until the sugar is dissolved

STEPS TO CONSTRUCT AN ALGORITHM

(i) Collect detailed information on the problem

Identify and understand the problem you want to solve. This involves

finding detailed information about it to enhance your understanding and help verify the steps needed for the solution.

(ii) Analyse the problem
Determine the inputs and outputs of the problem, and ensure the algorithm achieves the desired output.

(iii) Decide on the approach for solving the problem
Use a problem-solving method that best suits the problem. Choose an approach that reduces the effort and time required for execution.

(iv) Review the approach and better alternatives.
Evaluate the chosen approach to ensure it solves the problem efficiently. Compare it with alternative methods that may produce better results.

(v) Develop the entire organization of the algorithm
Outline and arrange all the necessary steps to solve the problem systematically.

(vi) Refine the algorithm.
After developing the algorithm, review and improve it for efficiency, clarity, and simplicity.

REPRESENTATION OF ALGORITHMS

Until now, algorithms have been represented as an outline of steps to solve a given problem. This approach suffices for algorithms to be carried out manually. Algorithms to be coded for computer based solutions are normally represented in different ways. That representation makes it easy to translate the algorithm into a computer program.

To address this need, the representation of algorithms is done using either flowcharts or pseudocode

FLOWCHARTS

A flowchart is a diagrammatic representation of an algorithm.

A flowchart is constructed using different types of symbols each representing a particular step or action taken in an algorithm.

One advantage of flowcharts is that they are helpful in detailed representation of algorithms.

Advantages of flowcharts

(i) ***Practical analysis***: The problems can be analysed more effectively using flowcharts
(ii) ***Efficient coding***: Flowcharts act as a guide or blueprint during the system analysis and program development phase You can easily convert flowcharts into code in any programming language
(iii) ***Proper documentation***: Flowcharts serve as good program documentation and are required for various purposes

Disadvantages of flowcharts

Although flowcharts are a convenient tool for representing algorithms, their uses has few limitations

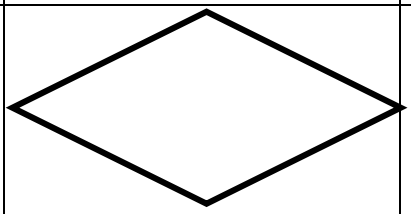
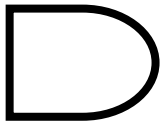
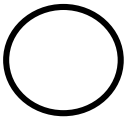
(i) ***Alterations and modifications:*** Changes can become problematic since they usually require rewriting the entire flowchart, which costs money and other resources.
(ii) ***Loss of detail:*** The essentials of what is done can easily lost in the technical details of how it is represented.
(iii) ***Complex logic:*** Flowcharts are not suited for representing complex algorithms.

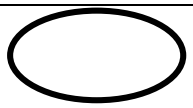
Rules for drawing flowcharts

The following six rules should be followed when creating program flowcharts:

(i) Only the standards symbols shown in the table below should be used in program flowcharts
(ii) There should be a logical flow of steps
(iii) Each symbol used in a program flowchart should contain only one entry point and one exit point except the decision symbol
(iv) The operations shown within a symbol of a program flowchart should be expressed independently of a particular programming language
(v) All decision branches should be labelled clearly:
(vi) All components of a flowchart should be connected through arrows

SYMBOLS USED IN FLOWCHARTS

S/N	***Symbol***	***Description***
1.		Process symbol Indicates the processing step. The description of a step is written inside the box This indicates any type of internal operation inside the processor or memory
2.		Flow lines symbol The arrow shows the direction of flow from one step or decision to another
3.		Decision symbol: This is the symbol used in a flowchart to indicate a point at which a decision must be made and a branch to one or more alternative points is possible. The decision symbol usually contains two exits: one labelled YES or TRUE and the other NO or FALSE. The decision is based on a question. The question is written inside the symbol
4.		Delay or wait symbol In flowcharts the “wait” or “delay” symbol indicates a temporary pause or halt in the execution of a process. It represents a specified period where the flowchart stops before moving on to the next step or action
5.		Connector symbol Specifies a link to another page or another flowchart when a complex flowchart is to be drawn with various flow lines directions. the same symbol on the other page indicates that the flow continues there

6.		Input or output Used for both input(read) and output(write) to denote any function of an I/O devices
7.		Printed document: Specifies if the input or output from the process is in the form of the printed document
8.		Terminal symbols: These are alternatives for start and end points. They always begin and end a flowchart. The start symbol can have only one flow line but no entering flow line. The stop symbol can have an entering flow line but no exit

These are a few symbols that are commonly used but you may come across other symbols depending on the complexity of the algorithm to be represented.

Steps for drawing a flowchart

There are various steps in drawing a flow chart. The following are general steps:

(i) ***Know the purpose of your flowchart***
Clearly understand the algorithm you want to represent in the flowchart. Identify the start and end points, the major steps in between.

(ii) ***Identify the steps***
Break down the process into individual steps or actions. Each step should represent a specific task or decision

(iii) ***Collect information***
Collect all the information and details needed to represent the algorithm accurately. This may include data inputs, decision points, actions and outputs

(iv) ***Add shapes and symbols:***

Use common symbols, including:

(a) Rectangle for processes or actions

(b) Diamond for decision points or conditional statements

(c) Ellipse for the start or end of the process

(d) Parallelogram for input or output of data

(e) Arrows connect symbols to show the flow of the process

(v) ***Determine the flow***

Decide the sequence of actions and decisions in the process. Determine which symbols to use for each step and how they connect. Shapes are connected using lines and arrows.

(vi) ***Add decision points***

If there are decision points in the process where different points can be taken based on certain conditions, add decision symbols (usually diamond shaped) to represent these points.

(vii) ***Include start and end points***

Include a starting symbol (usually an oval shape) to represent the beginning of the process and an ending symbol (also an oval) to represent the conclusion of the process.

(viii) ***Add connectors***

Employ link point (small circles) to join different parts of the flowchart together, especially extends across multiple pages

Example 01

Write an algorithm for calculating the sum of any two numbers entered by the user and then draw a flowchart for it.

Solution:

Algorithm for adding two numbers:

Step 1: Start by identifying two numbers, e.g. N1 and N2.

Step 2: Input or write the first number N1

Step 3: Input or write the second number N2

Step 4: Add the number N1 and N2, i.e N1 + N2

Step 5: Give the answer or output the sum, i.e. Sum=N1+N2.

Step 6: End the process

The following are steps for drawing the flowchart that will calculate the sum of any two numbers:

Step 1: Identify the inputs (two numbers, for example, N1 and N2)

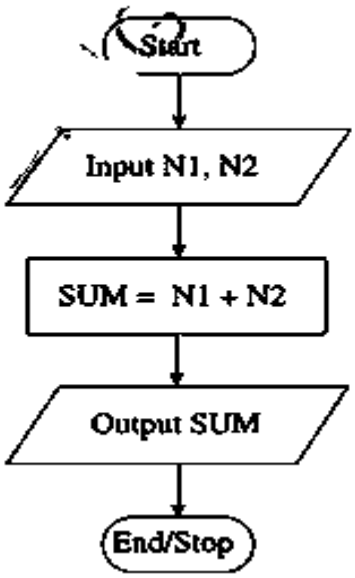
Step 2: Define the process or actions, that is, adding or calculating the sum of the two numbers

Step 3: Identify the output. Output is the sum

Step 4: Identify the standard symbols used in the problem

Step 5: Draw a flowchart, as shown in figure below

Flowchart for the sum of two numbers



Example 02

Suppose we want to write an algorithm top find the square of a number and represent it using a flowchart.

Before developing the algorithm, you should first identify the input, process, and output. In this case, input will be the number whose square is required.

The process will be to multiply the number by itself to get its square. The output will be the square of the number.

The following is the algorithm to find the square of a number

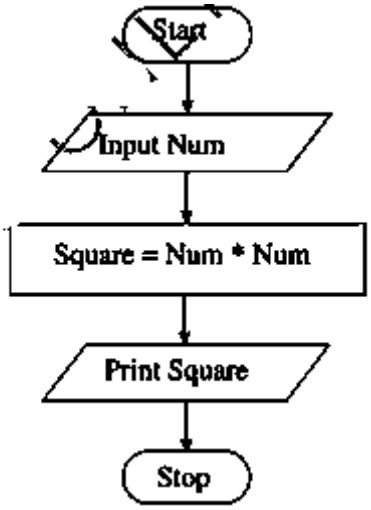
1. Get the number to be squared
2. Multiply the number by itself
3. Show the result

The steps for representing this algorithm in a flowchart are:

Step 1: Identify input. Input here is a number that is entered, 'Num'

Step 2: Define the process or action. The action here is to multiply where you compute Num*Num, that is a product output for Square.

Step 3: Determine or identify output. The output here is to Print Square. The flowchart will appears as shown in Figure below



REPRESENTATION OF ALGORITHMS USING PSEUDOCODE

Pseudocode is the textural representation of an algorithm containing a detailed description of instructions.

Pseudocode is a non-formal language that helps programmers to write algorithms.

It is a type of structured English or Swahili used to specify an algorithm.

It is neither compiled nor executed by a computer.

There are no accurate formatting or syntax rules for writing pseudocodes.

THE IMPORTANCE OF PSEUDOCODE

Pseudocode is easy to read, understand, and modify. Pseudocode helps review the steps to confirm that the expected implementation will achieve the desired output.

Advantages of pseudocode

The following are the advantages of pseudocode

(i) It can be easily modified unlike flowcharts.
(ii) It can be read and understood easily
(iii) It is easier to convert the pseudocode to a programming language than flowcharts
(iv) It is helpful in structured design.
(v) It can be easily written using any word processor or text editor.

Disadvantages of pseudocode

The following are the disadvantages of pseudocode

(i) It is more difficult for a beginner to follow the logic or write pseudocode than a flowchart.
(ii) There is no standard way of writing pseudocode. As a result we can have different pseudocodes from one problem.

In pseudocode, keywords are used to indicate common input, processing operations, and output. They can be written entirely in uppercase or start with a capital letter. The commonly used keywords are described in the table below

STANDARD KEYWORDS USED IN PSEUDOCODE

Item	Keyword	Description
1.	START or BEGIN	This is used whenever you start your pseudocode
2.	INPUT, READ, or GET	These are keywords used for inputting data.
3.	PRINT, DISPLAY, or SHOW	This will show your output to a screen or any output device
4.	COMPUTE, CALCULATE or DETERMINE	This is used to calculate the results of an expression

5.	SET	It is used to initialise values or assign a value to a variable
6.	INCREMENT	It is used when you want to increase the value of a variable by one
7.	DECREMENT	This keyword is meant to reduce the value of a variable by one
8.	END	Used to indicate the endpoint of the program or a process

RULES FOR WRITING PSEUDOCODE

(i) Only one statement should be written for each line, where each statement should represent a single action

(ii) The input, output, and process statements must be clearly stated using standard keywords, such as READ, PRINT, WRITE, DISPLAY, and RETURN. For example

```
START
        READ a, b, c
        c = a + b
        PRINT c
END
```

(iii) Each initial keyword should be capitalized. For example pseudocodes are used to calculate the area of a triangle.

```
START
        READ base, height, area
        COMPUTE: area=0.5*base*height
        PRINT area

END
```

(iv) Show indentation for the pseudocode. For example, the pseudocodes used to find the minimum of the two numbers, can be indicated as follows.

```
BEGIN
    READ num1, num2
If (num1<num2)
    SET Min=num1
```

Else

SET Min = num2
PRINT Min

END

(v) Pseudocodes should clearly show the start and end of executable statements and control structures.

(vi) The statements should be short, clear, and readable.

Example

Suppose you are given a task to find the sum and average of two numbers entered by user. Write pseudocode for this task and display the output on the screen

Procedure:

(i) Think about what your input will be, for example, let the two numbers be X and Y

(ii) Identify the process. In this case processes are add the X and Y numbers and divide the sum by 2 to get the average.

(iii) Identify the outputs. In this case the output is AVERAGE.

The pseudocode for calculating the sum and average will be as follows

START

PRINT" Enter two numbers."

INPUT X, Y

$SUM = X$+Y

AVERAGE= SUM/2

PRINT SUM

PRINT AVERAGE

END

COMPARISON BETWEEN FLOWCHAARTS AND PSEUDOCODE

The table below shows the comparison between flowcharts and pseudocode

S/n	Criteria	Pseudocode	Flowchart
1	Representation	Textural representation of an algorithm	Graphical representation of an algorithm
2	Level of details	It is flexible to accommodate many details	If many details are needed the flowchart becomes more complex
3	Alteration	The pseudocode is flexible to alteration after it has been designed	It is difficult to modify once it has already been designed
4	Format	There are no specific symbols to use, only textural description	Uses specific symbols to present the algorithm

CONTROL STRUCTURES IN FLOWCHARTS AND PSEUDOCODE

Control structures are essential components in programming that dictate how an execution of a program flows.

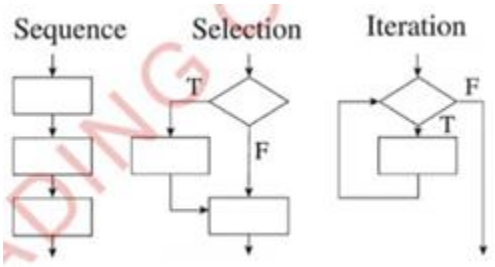
They assist programmers in controlling the order in which instructions are executed, enabling them to make decisions and repeat actions as needed.

There are three types of control structures that flowcharts and pseudocode operations employ.

The types of control structures are

(i) Sequence
(ii) Selection
(iii) Iteration

The figure below shows the control structures

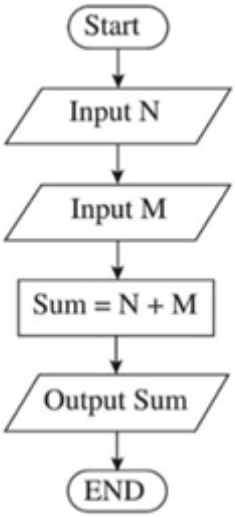


SEQUENCE CONTROL STRUCTURE IN FLOWCHARTS

The computer reads instructions from a program file line by line sequentially, starting from the first line. Examples from real life include adding numbers or any action with a series of steps before termination.

Consider your dress before leaving for school. There are approximately four phases that make up a sequence structure: putting on underwear, putting on trousers or skirt, putting on socks, and putting on a pair of shoes.

The following example illustrates the flowchart using a sequential control structure for an algorithm to add two numbers as shown in the figure below



SELECTION CONTROL STRUCTURE IN FLOWCHARTS

The selection control structure is used to make decisions on an algorithm.

A condition is a logical expression or statement that can be assessed as true or false.

The ability to control flow of the algorithm by making decisions based on conditions is provided by selection control structures.

It enables choosing between several paths of execution.

Examples from real life include deciding what to wear based on the weather. You dress in a T-shirt and shorts when it is sunny, but you take an umbrella and a raincoat when it is raining.

Under the selection control structure, one condition is evaluated first and then the program flows into one of the two or more paths.

Once the conditional is finished, it will re-join before leaving the structure.

Selection control structure uses 'IF… THEN' or 'IF… THEN … ELSE' or a CASE structure in pseudocodes and flowcharts.

IF…. THEN

IF…. THEN, then structure executes the process if the condition is true. If it is not true, it skips the process.

The diamond shaped box in flowcharts represents the decision or condition.

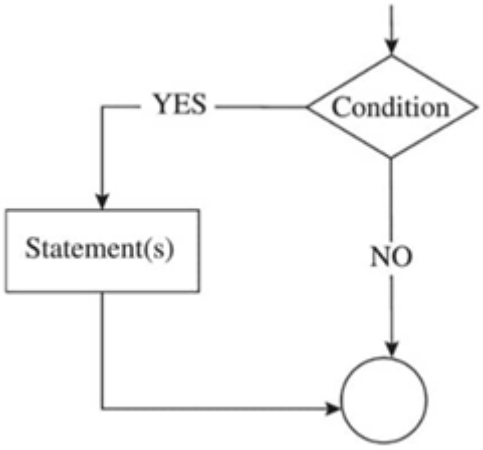
The syntax for IF…THEN is

IF (Condition) THEN

Statement

END IF:

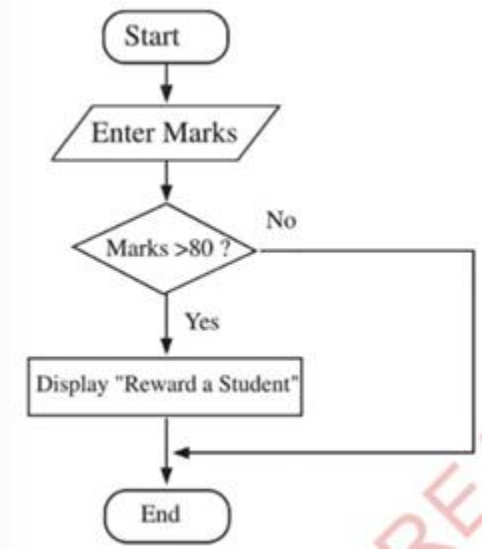
The corresponding flowchart is represented as shown in the figure below



Example of IF… THEN selection control structure

Write an algorithm using a flowchart for the headmaster of Mihayo secondary school who wants to reward students who attain an average mark above 80%

The figure below shows an example of a flowchart for rewarding students.



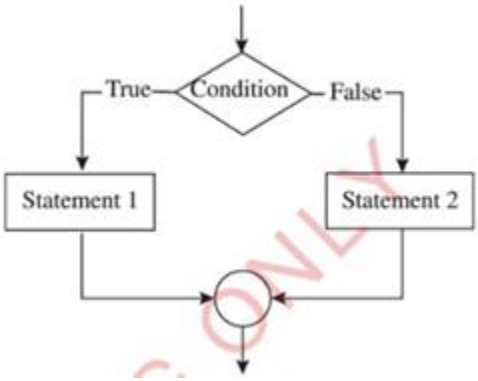
IF…THEN… ELSE

In this structure, if the condition is true, it executes statement 1. If the condition is false, it will execute the second process (statement 2). In this case, process 1 or 2 will only be implemented depending on the given condition.

The syntax of IF…THEN… ELSE is shown as follows:

```
IF (condition) THEN
        Statement 1
ELSE
        Statement 2
END IF
```

The corresponding flowchart is represented as shown in the figure below



When the IF-THEN- ELSE structure is encountered:

(i) The condition used to make the decision is tested

(ii) If the condition is true, the statements between I and THEN are executed. The FALSE part is skipped. And execution continues with the statement following ENDIF

(iii) If the condition is false, the THEN part is skipped. Instead, the statements between ELSE and ENDIF (i.e. the ELSE part of the structure) are executed, and execution continues with the statement after ENDIF

Examples of IF-THEN-ELSE selection control structure

Write an algorithm using a flowchart that reads two scores of a student, determines the largest score, and print the largest score with the message that identifies it.

Algorithm: Determining the largest score.

Step 1: Read the first score and assign it to be score1.

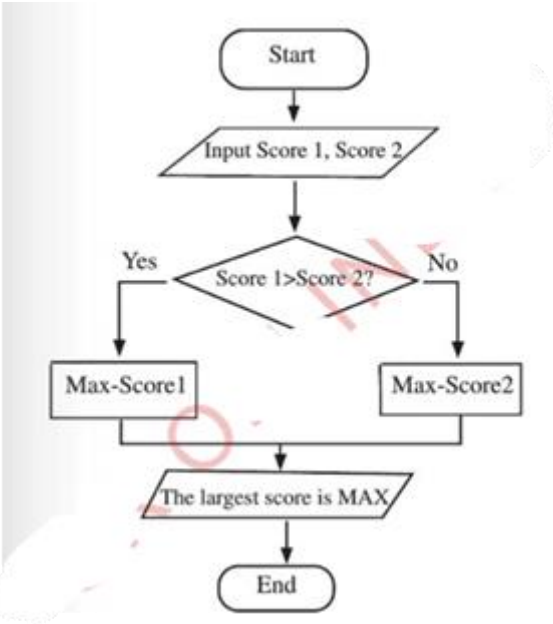
Step 2: Read the second score and assign it to be score2.

Step 3: If score1 is greater than score2, then display “The largest score is score1”

Step 4: Else, if score2 is greater than score1, then display “The largest score is score2”.

Step 5: END

The flowchart of IF-THEN –ELSE is shown in the figure below



IF….THEN….ELSE IF

You can introduce this IF statement if you want to select an action from several alternatives. This statement introduces the additional conditions. If condition1 is false, the *Else if* executes another condition. In this form of IF statement, more than one *Else if can be used.*

All conditions in the statement are evaluated one after another from top to bottom.

When one of them is true, then it will be executed. If all conditions are false, the sequence in *Else* clause is executed. The syntax for IF…..THEN…. ELSE IF is as follows

IF (condition1) **THEN**

Sequence_of_statements1

ELSE IF (condition2) **THEN**

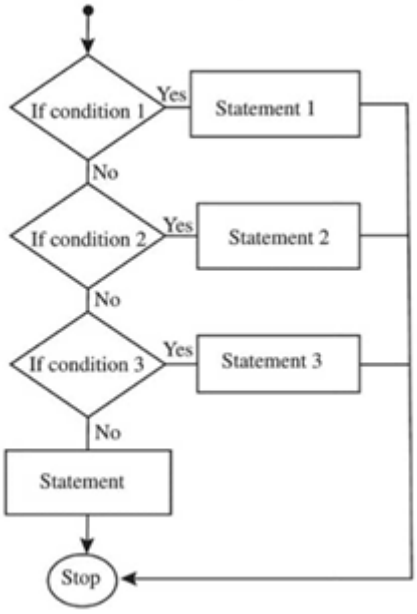
Sequence_of_sttatement2

ELSE

Sequence_of_statement3

END IF;

The corresponding flowchart is represented as shown in the figure below



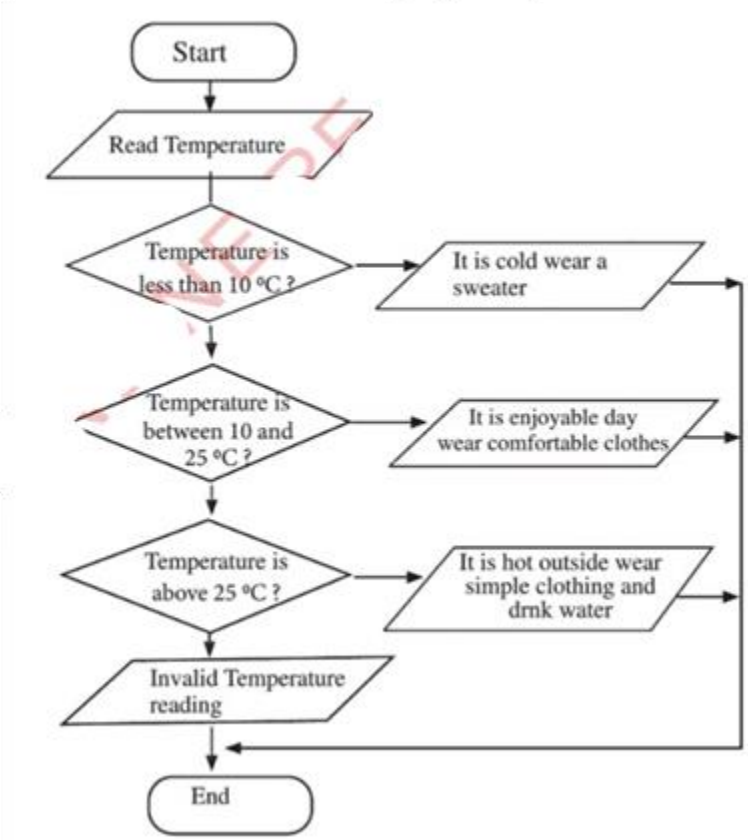
Example of IF-THEN-ELSEIF selection control structure

Write an algorithm to determine the weather conditions of your environment using a flowchart diagram.

Algorithm: Determining the weather conditions of your environment

(i) Read the current temperature
(ii) If the temperature is less than 10℃ , then display “It is cold, wear a sweater”
(iii) Else, if the temperature is between 10 to 25℃, then display “It is enjoyable day, wear comfortable clothes”
(iv) Else, display “Invalid temperature reading.”
(v) End

The flowchart of IF-THEN-ELSEIF is shown in the figure below



ITERATION CONTROL STRUCTURES IN FLOWCHARTS

Iteration also known as looping, means repeating steps or instructions over and over.

Iteration is used when there is a need to execute a statement or block of statement several times.

The repetition of loops is controlled with the help of a test condition. The statements in the loop are repetitive until the test condition becomes false. Examples from real life include:

(i) A cook in the kitchen may try different ingredients or adjust different steps of the cooking process slightly until the food tastes nice as required.

(ii) When a musician begins with one version of a song and improves it by modifying the speed or a few notes in this way, many different iterations of a song can be created up until the best final product has been produced that sounds and functions well.

(iii) A painter can also use iteration when creating artwork. The painter begins with a simple sketch and improves it by adjusting shapes, adding colours, or correcting mistakes. After each change, the painter reviews the drawing and then makes further improvements. By repeatedly modifying and refining the artwork in this way, many different versions of the painting are created until the best final masterpiece is produced that looks complete, attractive, and well-designed.

The loop structure consists of two main parts:
(a) Loop body- this represents the statements to be repeated
(b) Loop control – this specifies the entry/ exit condition of the loop.

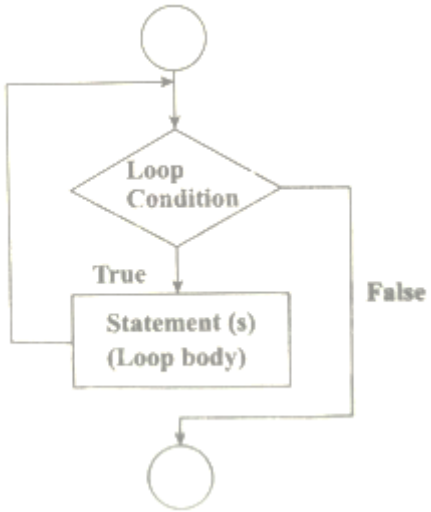
A loop is employed when you wish to repeat a set of instructions as long as a certain condition is true.

It consists of a condition that is checked before each iteration. If the conditions becomes false the loop terminates.

The general syntax of a loop is as follows:

```
(Condition) {
        Statement(s) to be executed;
}
```

The expression of the LOOP structure is illustrated in the figure below



Examples of loop iteration control structure

Consider that you have a "Kibubu" and that you wish to save money on it. Write an algorithm that asks the user to input coins in Tanzania shillings (Tshs) repeatedly and then adds the sum to the "Kibubu" until it exceeds or equals 6000 shillings. Construct this algorithm and draw a corresponding flowchart.

Algorithm:

(i) Start a loop with the condition that the Kibubu contains less than 6000.
(ii) Within the loop:
  (a) Prompt the user to input the amount of coins (in Tshs).
  (b) Add the input amount to **Kibubu**
  (c) Check if **Kibubu** is still less than or equal to 6000.
  (d) If the condition is false, exit the loop.
(iii) End the loop.
(iv) Display a message to the user that their Kibubu now contains 6000 shillings or more.
(v) End

The flowchart to implement the loop is presented in following figure

